

CHƯƠNG 2:

CÁC KỸ THUẬT CƠ BẢN

PHÂN TÍCH THUẬT TOÁN

Phân tích thuật toán là đưa ra các đánh giá về tài nguyên mà thuật toán đòi hỏi sử dụng. Đây là một công việc không đơn giản, nó đòi hỏi trình độ và kinh nghiệm. Chương này trình bày các kỹ thuật cơ bản để phân tích thuật toán ở khía cạnh thời gian tính.

2.1. Phân tích độ phức tạp của các cấu trúc lặp

2.1.1. Cấu trúc tuần tự.

Giả sử P và Q là hai đoạn của thuật toán, có thể là một câu lệnh nhưng cũng có thể là một thuật toán con. Gọi $Time(P)$, $Time(Q)$ là thời gian tính của P và Q tương ứng. Khi đó ta có

Quy tắc tuần tự: Thời gian tính đòi hỏi bởi “ $P; Q$ ”, nghĩa là P thực hiện trước, tiếp đến là Q , sẽ là

$$Time(P; Q) = Time(P) + Time(Q),$$

hoặc trong ký hiệu Theta:

$$Time(P; Q) = \Theta(\max(Time(P), Time(Q))).$$

Mặc dù qui tắc này rất đơn giản, nhưng khi áp dụng nó nhiều khi không đơn giản như ta nghĩ. Chẳng hạn, khi thời gian tính của Q phụ thuộc vào các thông số là các kết quả thực hiện của P . Vì vậy, để áp dụng được qui tắc này, cần phải có điều kiện là P và Q là độc lập.

2.1.2. Vòng lặp for.

Xét vòng lặp

```
for i := 1 to m do P(i);
```

Trước hết xét trường hợp đơn giản nhất khi thời gian tính của $P(i)$ không phụ thuộc vào i , mặc dù nó có thể phụ thuộc dữ liệu. Khi đó nếu thời gian thực hiện $P(i)$ là t thì thời gian hoàn thành vòng lặp là $m \times t$. Mặc dù cách tiếp cận này thông thường là chấp nhận được, nhưng ở đây tiềm ẩn một cái bẫy: Chúng ta đã không tính đến thời gian tổ chức vòng lặp for. Mỗi lần lặp của vòng for đòi hỏi một phép toán tăng chỉ số $i := i + 1$, một phép so sánh “ $i \leq m$ ” và một phép toán goto chuyển điều khiển.

Phân tích vòng lặp for sẽ phức tạp hơn, khi thời gian tính của $P(i)$ phụ thuộc vào i và là $t(i)$. Khi đó nếu không tính đến thời gian tổ chức vòng lặp for thì thời gian tính của vòng lặp

for $i := 1$ **to** m **do** $P(i)$;

sẽ là $\sum_{i=1}^m t(i)$.

Ví dụ. Xét thuật toán tính số Fibonacci

$$f_n = f_{n-1} + f_{n-2}, n \geq 2,$$

$$f_0 = 0, f_1 = 1.$$

```
function Fibiter(n)
begin
    i:=0; j:=1;
    for k:=1 to n do
    begin
        j:= j+i;
        i:= j-i;
    end;
    Fibiter:= j;
end;
```

Nếu coi các phép toán số học đòi hỏi thời gian là hằng số c , và không tính đến chi phí tổ chức vòng lặp for thì thời gian tính của hàm trên là $\Theta(n)$. Do số Fibonacci là rất lớn khi n là lớn. Chẳng hạn sử dụng công thức Muavre số hạng thứ k của dãy số sẽ là

$$f_k = \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^k - \left(\frac{1-\sqrt{5}}{2} \right)^k \right).$$

Từ đó suy ra số bit để biểu diễn f_k sẽ là $\Theta(k)$. Việc cộng và trừ các số có k bit có thể thực hiện với thời gian $\Theta(k)$. Suy ra, thời gian tính của lần lặp k sẽ bị chặn bởi $c.k$, trong đó c là hằng số. Từ đó ta có thời gian tính của thuật toán sẽ là

$$\sum_{k=1}^n c.k = c \sum_{k=1}^n k = c \frac{n(n+1)}{2} = \Theta(n^2).$$

2.1.3. Độ qui

Phân tích thuật toán đệ qui nói chung là phức tạp hơn. Cách tiếp cận thường dùng để phân tích độ phức tạp của các giải thuật đệ qui là: Trên cơ sở theo dõi hoạt động của giải thuật cố gắng đưa ra công thức đệ qui để tính số phép toán của nó.

Ví dụ. Xét thuật toán đệ qui để tính số Fibonacci

```

function FibRec(n);
begin
    if n < 2 then FibRec:=n
    else FibRec:= FibRec(n-1)+FibRec(n-2);
end;

```

Gọi $T(n)$ là thời gian tính của việc thực hiện lệnh gọi FibRec(n). Nếu $n < 2$ thuật toán chỉ thực hiện phép gán và trả lại giá trị hàm là n , điều đó đòi hỏi thời gian hằng số c . Ngược lại, công việc chính đòi hỏi là 2 lần gọi đệ qui đòi hỏi thời gian tương ứng là $T(n-1)$ và $T(n-2)$. Gọi $h(n)$ là thời gian thực hiện phép cộng và so sánh đòi hỏi trong lệnh gọi FibRec(n). Ta có

$$T(n) = \begin{cases} c, & n = 0, 1 \\ T(n-1) + T(n-2) + h(n), & n \geq 2. \end{cases}$$

Nếu ta giả thiết phép toán cộng và so sánh đòi hỏi thời gian hằng số thì $h(n)$ sẽ bị chặn bởi hằng số. Bằng qui nạp có thể chứng minh được rằng $T(n) = \Theta(f_n)$. Nếu để ý đến công thức Muavre, ta có thời gian này là hàm mũ của n .

2.1.4. Vòng lặp While và Repeat

Phân tích vòng lặp while và repeat gặp khó khăn chính là không biết tường minh từ đâu vòng lặp phải thực hiện bao nhiêu lần. Kỹ thuật cơ bản để phân tích các vòng lặp này là: Cần xác định một hàm của các biến trong vòng lặp sao cho hàm này có giá trị giảm dần trong quá trình lặp. Để chứng minh tính kết thúc của vòng lặp ta chỉ cần chỉ ra giá trị của hàm là số nguyên dương. Còn để xác định số lần lặp ta cần phải khảo sát xem giá trị của hàm giảm như thế nào.

Một cách tiếp cận khác để phân tích vòng lặp while là coi nó như một thuật toán đệ qui. Việc phân tích vòng lặp Repeat được tiến hành tương tự như phân tích vòng lặp While.

Ví dụ. Tìm kiếm nhị phân (Binary Search).

Cần tìm phần tử x trong mảng $T[1..n]$ với các phần tử được sắp xếp theo thứ tự không giảm. Giả sử x xuất hiện ít nhất một lần trong mảng T .

Thuật toán cần đưa ra chỉ số i : $1 \leq i \leq n$ sao cho $T[i] = x$.

```

function Binary-Search(T[1..n], x);
begin
    i:=1; j:=n;
    while i < j do
    begin
        k:= (i+j) div 2;
        case
            x < T[k]: j:=k-1;

```

```

        x = T[k]: i:=k; j:=k; Binary-Search=k; exit;
{Found!}

        x > T[k]: i:=k+1;

    end;

end;

end;

```

Cần xác định hàm của các biến số trong vòng lặp sao cho hàm này có giá trị giảm dần trong quá trình lặp. Ta xác định hàm $d = j - i + 1$. Như vậy d là số lượng phần tử của mảng T cần được khảo sát. Đầu tiên $d = n$. Vòng lặp kết thúc khi $i \geq j$, điều đó là tương đương với $d \leq 1$. Mỗi lần lặp có 3 khả năng có thể xảy ra: $j = k-1$; $i = k+1$; $i, j = k$.

Gọi d và d^* là giá trị của $j-i+1$ trước và sau khi thực hiện vòng lặp. Ta sử dụng i, j, i^*, j^* theo thứ tự để chỉ giá trị của i, j trước và sau khi thực hiện vòng lặp.

- Nếu $x < T[k]$, thì $i^*=i, j^*=(i+j) \div 2 - 1$, vì thế $d^*=j^* - i^* + 1 = (i+j) \div 2 - 1 - i + 1 \leq (i+j)/2 - i < (j - i + 1)/2 = d/2$.
- Nếu $x > T[k]$, thì $j^*=j, i^*=(i+j) \div 2 + 1$, vì thế $d^*=j^* - i^* + 1 = j - (i+j) \div 2 \leq j - (i+j-1)/2 - i = (j - i + 1)/2 = d/2$.
- Nếu $x = T[k]$, thì $d^* = 1$ còn $d \geq 2$.

Vậy ta luôn có $d^* \leq d/2$. Do vòng lặp phải dừng khi $d \leq 1$, nên thuật toán phải kết thúc.

Bây giờ ta đánh giá thời gian tính của thuật toán. Giả sử d_p là giá trị của $j - i + 1$ ở lần lặp thứ p , $p \geq 1$ và $d_0 = n$. Ta có

$$d_p \leq d_{p-1}/2, p \geq 1.$$

Suy ra

$$d_p \leq n/2^p.$$

Do vòng lặp phải kết thúc khi $d \leq 1$, nên điều đó phải xảy ra khi $p = \lceil \log n \rceil$. Ta đi đến kết luận là thuật toán kết thúc sau không quá $\lceil \log n \rceil$ lần lặp. Mỗi lần lặp đòi hỏi thời gian tính là hằng số. Vậy thuật toán có thời gian tính là $O(\log n)$.

Cách tiếp cận thứ hai để phân tích vòng lặp while là coi nó như một thuật toán đệ qui. Mỗi lần thực hiện vòng lặp chúng ta giảm được độ dài đoạn chứa x . Gọi $t(d)$ là thời gian nhiều nhất cần để thực hiện thuật toán khi $j-i+1 \leq d$, tức là khi còn phải xét d phần tử của T . Như đã chứng minh ở trên giá trị của $j-i+1$ giảm đi một nửa sau mỗi lần lặp, cho nên trong ngôn ngữ đệ qui điều này có nghĩa là $t(d)$ không vượt quá tổng thời gian thực hiện lần lặp (b) và thời gian $t(d \div 2)$ để đi đến kết thúc. Như vậy, ta có

$$t(d) \leq \begin{cases} c, & d = 1 \\ b + t(d \div 2), & d > 1. \end{cases}$$

Từ đó sử dụng kỹ thuật giải công thức đệ qui ta suy ra $t(n) = \Theta(\log n)$.

2.2. Câu lệnh đặc trưng

Trong việc phân tích nhiều thuật toán ta chỉ cần quan tâm đến việc đếm số lần thực hiện một câu lệnh đặc biệt nào đó. Câu lệnh đó được gọi là câu lệnh đặc trưng và cách phân tích như vậy được gọi là phân tích thuật toán sử dụng câu lệnh đặc trưng. Câu lệnh đặc trưng phải là câu lệnh được thực hiện thường xuyên ít nhất là cũng như bất kỳ câu lệnh nào trong giải thuật. Với giả thiết là thời gian tính của việc thực hiện các câu lệnh là bị chặn bởi hằng số, thời gian tính của thuật toán sẽ cùng cỡ với số lần thực hiện câu lệnh đặc trưng.

Cách phân tích này tỏ ra hữu ích, bởi vì nó cho phép ta bỏ qua thời gian tính của nhiều lệnh khác. Điều đó cho phép bỏ qua các hằng số đánh giá thời gian tính của việc thực hiện các phép toán cơ bản. Các hằng số này sẽ không có vai trò khi chúng ta đánh giá thuật toán trong ký hiệu tiệm cận.

Ví dụ. Xét thuật toán Fibiter.

Giả sử các phép tính số học là các phép tính cơ bản. Khi đó, để đánh giá thời gian tính của thuật toán, ta có thể coi câu lệnh “ $j := i + j$ ” là câu lệnh đặc trưng và dễ dàng đếm được số lần thực hiện nó là n . Vì vậy, thời gian tính của thuật toán là $\Theta(n)$.

Khi thuật toán đòi hỏi thực hiện nhiều vòng lặp lồng nhau, thì có thể lấy câu lệnh nằm trong vòng lặp trong cùng làm câu lệnh đặc trưng. Tuy vậy, cũng cần hết sức thận trọng khi sử dụng cách lựa chọn này.

Ví dụ. Sắp xếp kiểu nhốt chim vào chuồng (Pigeonhole Sorting).

```
procedure Pigeonhole-Sorting;  
for i:=1 to n do inc(U[T[i]]);  
{ Sắp xếp n số nguyên dương có giá trị nằm trong khoảng 1..s.  
  T[1..n]: mảng chứa dãy được sắp xếp.  
  U[1..s]: là mảng mà phần tử U[k] đếm số lượng số có giá trị là k trong mảng T.  
  Việc “đếm chim” đã được thực hiện trước đó để có được mảng U[ ].  
  Thủ tục dưới đây chỉ thực hiện khâu “nhốt chim vào chuồng”}
```

```
begin  
  i:=0;  
  for k:=1 to s do  
    while U[k]<>0 do  
      begin  
        i:=i+1;  
        T[i]:=k;  
        U[k]:=U[k]-1;  
      end;  
    end;
```

end;

Nếu chọn câu lệnh bất kỳ trong vòng lặp while làm câu lệnh đặc trưng, thì rõ ràng với mỗi k , câu lệnh này phải thực hiện $U[k]$ lần. Do đó số lần thực hiện câu lệnh đặc trưng là $\sum_{k=1}^s U[k] = n$ (do có tất cả n số cần sắp xếp). Từ đó ta có thời gian tính là $\Theta(n)$.

Ví dụ sau đây cho thấy đánh giá đó chưa chính xác. Giả sử $T[i] = i^2, i = 1, \dots, n$. Khi đó

$$U[k] = \begin{cases} 1, & \text{nếu } k \text{ là số chẵn} \\ 0, & \text{nếu } k \text{ là số lẻ} \end{cases}$$

Khi đó $s = n^2$. Rõ ràng thời gian tính của thuật toán không phải là $O(n)$, mà phải là $O(n^2)$. Lỗi ở đây là do ta xác định câu lệnh đặc trưng chưa đúng, các câu lệnh trong thân vòng lặp while không thể dùng làm câu lệnh đặc trưng trong trường hợp này, do có rất nhiều vòng lặp rỗng (khi $U[k] = 0$). Nếu ta chọn câu lệnh kiểm tra $U[k] \neq 0$ làm câu lệnh đặc trưng thì rõ ràng số lần thực hiện nó sẽ là $n + s$. Vậy thuật toán có thời gian tính $O(n+s) = O(n^2)$.

2.3. Phân tích khấu trừ (Amortized Analysis)

2.3.1. Đặt vấn đề

Phân tích tình huống tồi nhất đôi khi cho kết quả không sát. Chẳng hạn, xét thuật toán A trong đó đòi hỏi thực hiện một dãy gồm n thao tác P đối với một cấu trúc dữ liệu I:

Thuật toán A

for $i:=1$ to n do P(I);

Ta cần đánh giá thời gian tính của thuật toán. Để đánh giá thời gian tính tồi nhất của thuật toán, ta sẽ coi P như là một thuật toán con và đánh giá thời gian tính tồi nhất của nó, chẳng hạn thời gian đó là t . Sau đó, thời gian tính của thuật toán trong tình huống tồi nhất sẽ được đánh giá bởi

$$\langle \text{số thao tác cần thực hiện} \rangle \times t = n \times t.$$

Đánh giá này có thể sẽ rất không sát với thời gian tính thực tế của thuật toán A, khi mà chỉ có một số rất ít các thao tác đòi hỏi thời gian tính lớn như vậy, còn phần lớn các thao tác lại có thể thực hiện rất nhanh. (Điều đó có thể xảy ra khi thao tác P thực hiện đối với cấu trúc dữ liệu I làm biến đổi cấu trúc này.) Vì vậy, trung bình mà xét, có thể coi các thao tác đều đòi hỏi thời gian là không lớn. Trong trường hợp này, để đánh giá thời gian tính của thuật toán ta cần biết cách tính thời gian trung bình thực hiện một thao tác trong cả dãy thao tác cần thực hiện.

Định nghĩa. Giả sử ta cần thực hiện một dãy các thao tác đối với cấu trúc dữ liệu. Gọi $T(n)$ là thời gian lớn nhất để thực hiện dãy n thao tác đối với cấu trúc dữ liệu. Khi đó **thời gian tính khấu trừ** cho một thao tác sẽ là $T(n)/n$.

Một cách tương đương, nếu ta biết thời gian tính khấu trừ là $U(n)$, thì mọi dãy gồm n thao tác có thể thực hiện với thời gian $n U(n)$.

Vấn đề đặt ra là làm thế nào có thể tính được thời gian tính khấu trừ. Dưới đây ta trình bày các kỹ thuật cơ bản để giải quyết vấn đề này.

2.3.2. Ví dụ: Cài đặt con đếm nhị phân (Binary counter)

Chúng ta cần thực hiện hai thao tác sau đây đối với biến A :

1. *settozero*: $A := 0$; và
2. *increment*: $A := A + 1$.

Cấu trúc dữ liệu mà ta sử dụng là danh sách

$$\langle \dots, a_3, a_2, a_1, a_0 \rangle$$

chứa biểu diễn nhị phân của con đếm A . Tức là $a_i \in \{0, 1\}$ với mọi $i \geq 0$, và

$A = \sum_{i \geq 0} a_i 2^i$. Để ý rằng ta chỉ cần danh sách có độ dài $\lfloor \log A \rfloor + 1$ để biểu diễn A , bởi

vì $A = \sum_{i=0}^{\lfloor \log A \rfloor} a_i 2^i$. Như vậy, với mọi $i \geq \lfloor \log A \rfloor$ ta có $a_i = 0$. Bây giờ hai thao tác trên có thể cài đặt như sau.

1. *settozero*: Tạo danh sách $\langle a_i = 0, i = 0, \dots, n \rangle$.

Ta coi thời gian tính của thao tác này là 1, tức là thao tác *settozero* đòi hỏi thời gian tính là 1 đơn vị.

2. *increment*:

```

a0 := a0 + 1;
i := 0;
while ai = 2 do
begin
    ai := 0;
    ai+1 := ai+1 + 1;
    i := i + 1;
end;
```

Thời gian tính của thao tác này là 1 cộng với số lần lặp của vòng lặp while.

Giả sử n là số nguyên dương. Ta cần tính thời gian thực hiện của đoạn chương trình (thuật toán) *Bin_Count* sau đây

```

Thuật toán Bin_Count;
settozero;
```

for $k:=1$ to n do increment;

tức là thời gian tính của 1 lần thực hiện *settozero* và dãy n thao tác *increment*. Do thời gian tính của thao tác thứ 1 ta giả thiết là 1, nên ta chỉ còn phải tính thời gian của việc thực hiện n thao tác *increment*.

Trước hết nhận thấy rằng thời gian thực hiện *increment* thay đổi rất thất thường.

A		a_5	a_4	a_3	a_2	a_1	a_0
47	=	1	0	1	1	1	1
48	=	1	1	0	0	0	0
49	=	1	1	0	0	0	1

Để tăng con đếm từ 47 lên 48 vòng lặp *while* phải thực hiện 4 lần lặp vì thế thời gian tính của *increment* là 5 đơn vị, trong khi đó để tăng từ 48 lên 49 nó lại không thực hiện một lần lặp nào cả và vì thế *increment* chỉ đòi hỏi 1 đơn vị thời gian.

Ta sẽ đưa ra đánh giá cận trên đầu tiên cho thời gian tính của thuật toán *Bin_Count*. Giả sử $0 \leq k \leq n-1$, xét lần thực hiện *increment* thứ k , tức là A sẽ tăng từ k lên $k+1$. Trong thao tác này vòng lặp *while* thực hiện không quá $\lfloor \log k \rfloor + 1$ lần lặp (ta định nghĩa $\lfloor \log 0 \rfloor = 0$). Vì vậy, thời gian tính của *Bin_Count* bị chặn trên bởi

$$\sum_{k=0}^{n-1} (\lfloor \log k \rfloor + 2) \leq n \log n + 2n.$$

Đây chỉ là một cận trên cho thời gian tính của *Bin_Count*. Đánh giá này là quá lớn. Dưới đây chúng ta sẽ thấy rằng *Bin_Count* có thời gian tính là $O(n)$. Từ đó theo định nghĩa, thời gian tính khấu trừ của *increment* là $O(n)/n = O(1)$. Điều đó có nghĩa là trong việc thực hiện dãy thao tác *increment*, chi phí thời gian trung bình cho việc thực hiện một thao tác này là hằng số.

2.3.2. Phương pháp thứ nhất: Tính toán chi li.

Trong cận trên thu được ở trên, chúng ta đã trả $\lfloor \log k \rfloor + 2$ cho lần tăng thứ k . Đối với một số giá trị của k , đây chính là thời gian cần thiết để thực hiện thao tác thứ k . Ví dụ, nếu k có dạng

$$k = 2^{j-1} + 2^{j-2} + \dots + 2^2 + 2^1 + 2^0,$$

thì có tất cả $j = \lfloor \log k \rfloor + 1$ lần lặp trong vòng lặp *while* của lần tăng con đếm thứ k . Vì thế thao tác này đòi hỏi thời gian $j + 1 = \lfloor \log k \rfloor + 2$. Tuy nhiên, đối với phần lớn các giá trị của k , lần tăng thứ k đòi hỏi ít thời gian hơn rất nhiều so với $\lfloor \log k \rfloor + 2$ đơn vị thời gian.

Chính xác hơn, nếu k là chẵn thì lần tăng thứ k chỉ đòi hỏi 1 đơn vị thời gian, bởi vì vòng lặp while không thực hiện một lần lặp nào. Trong quá trình thực hiện n thao tác tăng, tình huống này xuất hiện $\left\lceil \frac{n}{2} \right\rceil$ lần (vì k chạy từ 0 đến $n-1$).

Nếu $k \equiv 1 \pmod{4}$, thì vòng lặp while thực hiện một lần lặp, bởi vì con đếm tăng từ

$$\langle \dots, a_3, a_2, 0, 1 \rangle$$

thành

$$\langle \dots, a_3, a_2, 1, 0 \rangle.$$

Trong quá trình thực hiện n thao tác tăng tình huống này xuất hiện $\left\lceil \frac{n}{4} \right\rceil$ lần.

Tổng quát, giả sử $i \geq 0$. Trong quá trình thực hiện n thao tác tăng, có $\left\lceil n/2^{i+1} \right\rceil$ lần trong đó vòng lặp while thực hiện đúng i lần. Điều đó xảy ra khi con đếm tăng từ

$$\langle \dots, a_{i+2}, a_{i+1}, 0, 1, 1, \dots, 1 \rangle$$

với i số một tận cùng thành

$$\langle \dots, a_{i+2}, a_{i+1}, 1, 0, 0, \dots, 0 \rangle.$$

với i số 0 tận cùng. Từ đó suy ra, thời gian cần thiết để thực hiện n thao tác tăng sẽ là

$$\begin{aligned} & \sum_{k=0}^{n-1} (1 + \text{số lần tăng đúng } k \text{ lần trong } n \text{ thao tác tăng}) \\ & \leq n + \sum_{i=0}^{1+\lfloor \log(n-1) \rfloor} i \cdot \left\lceil n/2^{i+1} \right\rceil \\ & = O\left(n + \sum_{i=0}^{1+\lfloor \log(n-1) \rfloor} n \cdot \frac{i}{2^i}\right) \\ & = O\left(n + n \sum_{i=0}^{\infty} \frac{i}{2^i}\right) \\ & = O(n) \end{aligned}$$

đó chính là điều ta mong đợi.

Như vậy để có được kết quả chính xác chúng ta đã mất công tính toán kỹ lưỡng tổng thời gian phải trả chứ không ước lượng đại thể như đã làm ban đầu. Đó chính là ý tưởng của phương pháp thứ nhất để xác định thời gian khấu trừ.

2.3.3. Phương pháp thứ hai: Sử dụng hàm thế năng (Using potential function)

Với mỗi $0 \leq k \leq n-1$, gọi c_k là bằng 1 cộng với số lượng lần lặp của vòng lặp while trong lần tăng thứ k . Khi đó tổng thời gian thực hiện n thao tác tăng sẽ là

$$\sum_{k=0}^{n-1} c_k.$$

Vấn đề đặt ra bây giờ là tìm cách ước lượng tổng này.

Phương pháp thế năng là phương pháp thích hợp để tìm ra ước lượng đó. Phương pháp này khá dễ hiểu, tuy nhiên, không dễ áp dụng khi phải tự tìm lấy lời giải. Chúng ta sẽ bắt đầu với việc trình bày ý tưởng tổng quát của phương pháp. Sau đó sẽ minh hoạ áp dụng nó vào Thuật toán con đếm nhị phân.

Xét cấu trúc dữ liệu mà đối với nó ta thực hiện n thao tác. Cấu trúc dữ liệu ban đầu ký hiệu là D_0 . Với $k = 0, 1, \dots, n-1$, gọi c_k là chi phí thực hiện thao tác k , và gọi D_{k+1} là cấu trúc dữ liệu thu được sau khi thực hiện thao tác đối với D_k . Do đó tổng chi phí để

thực hiện n thao tác là $\sum_{k=0}^{n-1} c_k$.

Giả sử Φ là hàm đặt tương ứng mỗi cấu trúc dữ liệu D_k với số thực $\Phi(D_k)$, $0 \leq k \leq n$. Hàm này được gọi là hàm thế năng. $\Phi(D_k)$ được gọi là thế năng tương ứng với D_k .

Cho trước hàm Φ , ta định nghĩa

$$\hat{c}_k = c_k + \Phi(D_{k+1}) - \Phi(D_k), 0 \leq k \leq n-1.$$

Như vậy, $\hat{c}_k$ là tổng của chi phí thực hiện thao tác k và lượng giảm thế năng ở bước này. Ta có thể viết lại tổng chi phí của n thao tác như sau

$$\begin{aligned} \sum_{k=0}^{n-1} c_k &= \sum_{k=0}^{n-1} (\hat{c}_k - \Phi(D_{k+1}) + \Phi(D_k)) \\ &= \Phi(D_0) - \Phi(D_n) + \sum_{k=0}^{n-1} \hat{c}_k. \end{aligned}$$

Mục đích của chúng ta là đánh giá tổng $\sum_{k=0}^{n-1} c_k$, một nhiệm vụ có thể là khó khăn do

khó xác định được giá trị của c_k . Giả sử rằng hàm thế năng thoả mãn

$$(P.1) \quad \Phi(D_0) = 0 \text{ và}$$

$$(P.2) \quad \Phi(D_k) \geq 0 \text{ với mọi } 0 \leq k \leq n.$$

Khi đó

$$\sum_{k=0}^{n-1} c_k = \Phi(D_0) - \Phi(D_n) + \sum_{k=0}^{n-1} \hat{c}_k \leq \sum_{k=0}^{n-1} \hat{c}_k,$$

nghĩa là tổng $\sum_{k=0}^{n-1} \hat{c}_k$ là cận trên cho tổng chi phí thực hiện n thao tác.

Chú ý là điều đó là đúng với mọi hàm thế năng Φ thoả mãn (P.1) và (P.2). Điều cần bàn là tìm cách xác định hàm thế năng Φ sao cho tổng $\sum \hat{c}_k$ có thể tính được hoặc ít ra là ước lượng được một cách dễ dàng.

Chú ý: Chi phí khấu trừ cho một thao tác là

$$\frac{1}{n} \sum_{k=0}^{n-1} c_k \leq \frac{1}{n} \sum_{k=0}^{n-1} \hat{c}_k$$

Trong nhiều ứng dụng, giá trị của c_k có thể biến thiên hỗn loạn, trong khi đó $\hat{c}_k$ lại gần như là bằng nhau với mọi k . Trong tình huống đó, chi phí khấu trừ bị chặn bởi $\hat{c}_k$. Vì thế $\hat{c}_k$ được gọi là chi phí khấu trừ cho thao tác k .

Bây giờ ta xét cách áp dụng phương pháp thế năng đối với bài toán con đếm nhị phân. Trong trường hợp này, D_k là danh sách các bit của biểu diễn nhị phân của con đếm A . Chúng ta đã thấy chi phí c_k cho thao tác tăng thứ k là bằng tổng của 1 với số lần lặp của vòng lặp while. Nếu ngay trước thao tác này con đếm có dạng

$$\langle \dots, a_{i+2}, a_{i+1}, 0, 1, 1, \dots, 1 \rangle$$

với i số 1 tận cùng, thì $c_k = 1 + i$.

Ta định nghĩa hàm thế năng Φ như sau. Giả sử

$$\langle \dots, a_3, a_2, a_1, a_0 \rangle$$

là cấu trúc dữ liệu D_k . Khi đó

$$\Phi(D_k) := \sum_{j \geq 0} a_j,$$

là số lượng số 1 trong biểu diễn nhị phân của con đếm.

Trước hết nhận thấy rằng, $\Phi(D_0) = 0$ bởi vì ta bắt đầu từ con đếm $A = 0$. Tiếp theo, rõ ràng $\Phi(D_k) \geq 0$ với mọi $0 \leq k \leq n$. Như vậy cả hai đòi hỏi (P.1) và (P.2) đều được thực hiện. Còn giá trị của $\hat{c}_k$ thì sao? Ta có, theo định nghĩa

$$\hat{c}_k = c_k + \Phi(D_{k+1}) - \Phi(D_k).$$

Ngay trước khi thực hiện thao tác tăng thứ k , con đếm có dạng

$$D_k = \langle \dots, a_{i+2}, a_{i+1}, 0, 1, 1, \dots, 1 \rangle$$

với i số 1 tận cùng. Khi đó thao tác tăng thứ k sẽ cho kết quả

$$D_{k+1} = \langle \dots, a_{i+2}, a_{i+1}, 1, 0, 0, \dots, 0 \rangle,$$

với i số 0 tận cùng. Ta đã nhận thấy rằng, $c_k = 1 + i$. Rõ ràng $\Phi(D_{k+1}) - \Phi(D_k) = 1 - i$. Do đó

$$\hat{c}_k = (1 + i) + (1 - i) = 2.$$

Từ đó suy ra

$$\sum_{k=0}^{n-1} c_k \leq \sum_{k=0}^{n-1} \hat{c}_k = \sum_{k=0}^{n-1} 2 = 2n.$$

Vì vậy, tổng thời gian thực hiện n thao tác là

$$\sum_{k=0}^{n-1} c_k \leq 2n.$$

hay một cách tương đương có thể nói thời gian khấu trừ cho việc thực hiện một thao tác tăng bị chặn trên bởi 2. Để ý rằng việc lựa chọn hàm thế năng Φ đã thực sự làm cho việc tính $\sum \hat{c}_k$ trở nên dễ dàng hơn.

Chú ý. Điểm khó trong việc áp dụng phương pháp thế năng là xác định được trùng hàm thế năng Φ . Khi đã chọn được trùng hàm thế năng thì phần còn lại được thực hiện khá dễ dàng.